SQLite

SQLite is a self-contained, file-based SQL database.

SQLite is not directly comparable to client/server SQL database engines such as Oracle, PostgreSQL or MySQL, it is intended to provide local data storage for individual applications and devices.

SQLite comes bundled with Python and can be used in any Python applications without having to install any additional software.

In [2]:
```
import sqlite3
from sqlite3 import Error
```

Creating a connection to a SQLite database

The sqlite3.connect() function returns a Connection object that can be used to interact with the SQLite database (that ultimately resides in a file). The database (file) will be created automatically if it does not exists.

In [4]:
```
db_file= "./journal_articles.db"

connection = None
try:
    connection = sqlite3.connect(db_file)
    print(sqlite3.version)
except Error as e:
    print(e)

if connection is None:
    print("Cannot create the database connection.")
```

```
2.6.0
```

Creating tables

In order to execute SQL statements like CREATE TABLE, you need to first create a Cursor object, by calling the cursor() method of the Connection object, and then pass the SQL statement to the execute() method of the Cursor object.

In [5]:
```
def create_table(conn, create_table_sql):
    """ Create a table from a CREATE TABLE SQL statement
    Parameters
    ----------
    conn: Connection object
    create_table_sql: a CREATE TABLE statement
    """
    try:
        cursor = conn.cursor()
        cursor.execute(create_table_sql)
    except Error as e:
        print(e)
```

In [6]:
```
sql_create_journal_table = """
CREATE TABLE IF NOT EXISTS journal (
    journal_id integer PRIMARY KEY,
    journal_name text NOT NULL,
    publisher text DEFAULT 'Springer Nature' NOT NULL
    );
"""

if connection is not None:
    # Create journal table
    create_table(connection, sql_create_journal_table)
else:
    print("A valid connection object is required.")
```

Inserting data

To use a variable in a SQL statement, write a question mark (?) as a placeholder for each argument, and substitute the actual values by providing them as a tuple of values to the cursor's execute() method.

In SQLite, a column with type INTEGER PRIMARY KEY is an alias for the ROWID, a permanent unique identifier for a row. On an INSERT, if the INTEGER PRIMARY KEY column is not explicitly given a value, then it will be filled automatically with an unused integer, usually one more than the largest ROWID currently in use.

In [13]:
```
def insert_new_journal(conn, journal):
    """
    Insert a new journal into the journal table
    Parameters
    ----------
    conn: Connection object
    journal: Values to bind to placeholders in sql statement (a tuple) .
    Returns
    -------
    the row id of the inserted row.
    """
    cursor = conn.cursor()
    # For the question marks style, parameters must be a sequence e.g. a tuple
    cursor.execute("INSERT INTO journal(journal_name, publisher) VALUES(?,?)", journal)
    conn.commit()
    return cursor.lastrowid
```

In [14]:
```
if connection is not None:
    journal = ('Neurocomputing', 'Elsevier');
    journal_id = insert_new_journal(connection, journal)
    print(f"journal_id: {journal_id}")

    journal = ('IEEE Access', 'IEEE');
    journal_id = insert_new_journal(connection, journal)
    print(f"journal_id: {journal_id}")
else:
    print("A valid connection object is required.")
```

```
journal_id: 8
journal_id: 9
```

Querying Data

The fetchall() method of the Cursor object fetches all the rows of a query result. It returns all the rows as a list of tuples. An empty list is returned if there is no record to fetch.

In [23]:
```
def select_journals_by_publisher(conn, publisher):
    """
    Query journals by publisher
    Parameters
    ----------
    conn: Connection object
    publisher: Value to bind to placeholder in sql statement.
    """
    cursor = conn.cursor()

    # For the question marks style, parameters must be a sequence e.g. a tuple
    cursor.execute("SELECT * FROM journal WHERE publisher=?", (publisher,))

    # Never use string operations to assemble queries, as they are vulnerable to SQL injection
    # sql = "SELECT * FROM journal WHERE publisher='%s'" % publisher
    # print(sql)
    # cursor.execute(sql)

    rows = cursor.fetchall()

    for row in rows:
        print(row)
```

In [24]:
```
if connection is not None:
    # publisher = 'Elsevier'
    publisher = "' OR TRUE; --"
    select_journals_by_publisher(connection, publisher)

else:
    print("A valid connection object is required.")
```

In [25]:
```
if connection:
    connection.close()
```

In [ ]:
```
%%shell
jupyter nbconvert --to html /content/SQLite.ipynb
```

```
[NbConvertApp] Converting notebook /content/SQLite.ipynb to html
[NbConvertApp] Writing 288730 bytes to /content/SQLite.html
```

Out[ ]:

In [ ]: